University of Hertfordshire UH

School of Physics, Engineering and Computer Science

Department of Physics, Astronomy and Mathematics

MSc Data Science Professional Team Project 7PAM2033

GROUP FINAL REPORT

Project Title

A Hybrid Meta-Learning Architecture for Insurance Fraud Detection: Integrating DistilBERT Textual Embeddings with Gradient Boosting

Team Members

Student Name	Student Number
Vrushali Patel	24158441
Prince Patel	24158566
Nimra Khizar	24142732

Team GitHub Link:
https://github.com/orgs/Professional-Team-Project-Group-25/repositories

Supervisor: Paurush Punyasheel

Date Submitted: 24th July 2026

Word Count: Approximately 6680 words (excluding front page, declaration, acknowledgements, references and appendices)

DECLARATION STATEMENT

Any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6)

Human participants were not used in this MSc Project.

I have read the detailed guidance on academic integrity, misconduct and plagiarism and understand the University process for dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project or course.

Permission is given for the report to be made available on module websites provided the source is acknowledged.

Declaration of Use of AI

- **Yes** – AI tools have been used in the preparation of this project and report
    - **Yes** – To generate initial ideas.
    - **No** – To develop code.
    - **No** – To provide literature suggestions and summaries.
    - **Yes** – To provide suggestions to improve the report content.
    - **Yes** – To edit spelling and grammar, and to proofread the report.

Note: Copying text generated by AI into the report is academic misconduct.

Student Name	SRN Number	Signature	Date
Vrushali Patel	24158441	Vrushali	24/07/2026
Prince Patel	24158566	Prince	24/07/2026
Nimra Khizar	24142732	Nimra	24/07/2026

Acknowledgements

The team would like to express sincere gratitude to our supervisor **Paurush Punyasheel** for their guidance, feedback and continued support throughout the duration of this project.

We acknowledge the following datasets and tools used in this work:

- The insurance_claims.csv dataset, sourced from Kaggle (originally curated by Bunty Shah), which forms the primary experimental dataset for Experiments 2 and 3.
- The fraud_oracle.csv dataset (Oracle Vehicle Insurance Fraud Detection Dataset), sourced from Kaggle, used as the baseline tabular benchmark in Experiment 1.
- Hugging Face Transformers library (Wolf et al., 2020) for providing the pre-trained DistilBERT model and tokeniser used in the NLP pipeline.
- Scikit-learn (Pedregosa et al., 2011) for machine learning utilities including RandomForestClassifier, GridSearchCV, LabelEncoder, StandardScaler, and evaluation metrics.
- XGBoost (Chen and Guestrin, 2016) for the gradient boosted tree framework used across multiple experiments.
- TensorFlow/Keras (Abadi et al., 2015) for the deep learning infrastructure used to implement and train the BiLSTM architecture.
- Imbalanced-learn (Lemaître et al., 2017) for the SMOTE oversampling technique applied to address class imbalance.
- Streamlit for the interactive application demonstration dashboard (Streamlit Inc., 2023).
- Matplotlib (Hunter, 2007) and Seaborn (Waskom, 2021) for data visualisation.

AI writing assistance tools were used for grammar correction and proofreading only. No AI-generated text has been directly copied into this report.

Abstract

Insurance claim fraud is a pervasive and costly problem for the global insurance industry, estimated to cost the UK industry over £1.1 billion annually (ABI, 2023). This report presents the development, evaluation, and comparison of three experimental machine learning pipelines for automated insurance fraud detection, encompassing tabular baseline models, deep learning architectures, and a novel hybrid approach combining Natural Language Processing (NLP) with gradient boosting. Three structured experiments are conducted across two real-world datasets: the Fraud Oracle dataset (15,420 records, 6% fraud prevalence) and the Insurance Claims dataset (1,000 records, 24.7% fraud prevalence). Experiment 1 establishes tabular baselines using Random Forest and XGBoost on the Fraud Oracle dataset. Experiment 2 applies the same approaches to the Insurance Claims dataset, incorporating SMOTE oversampling to address class imbalance, and introduces a Bidirectional Long Short-Term Memory (BiLSTM) deep learning model. Experiment 3 constitutes the core research contribution: a novel hybrid pipeline that converts structured claim features into natural language narratives, scores them using a fine-tuned DistilBERT Transformer model, and uses the resulting fraud probability as an engineered feature within an XGBoost meta-learner. Results demonstrate that the Random Forest baseline achieves an F1-score of 0.51 and ROC-AUC of 0.84 on the Insurance Claims test set, outperforming the BiLSTM (F1: 0.44, AUC: 0.82) and the Hybrid RF+BiLSTM ensemble (F1: 0.46). The Experiment 3 hybrid architecture demonstrates the feasibility of NLP-augmented fraud detection, with the DistilBERT score confirmed as an informative feature via XGBoost feature importance analysis. A production-ready Streamlit dashboard enables interactive claim triage with risk-tiered probability outputs. Findings contribute to the growing body of literature on applied NLP in financial fraud detection and highlight transfer learning as a viable strategy for extracting semantic fraud signals from structured claim data.

Table of Contents

1. Introduction
   1.1 Research Question
   1.2 Research Objectives
   1.3 Report Structure
2. Data
   2.1 Dataset Selection
   2.2 Dataset Descriptions
   2.3 Exploratory Data Analysis
   2.4 Pre-processing and Feature Engineering
   2.5 Data Ethics Statement
3. Methodology
   3.1 Research Design and Experimental Framework
   3.2 Evaluation Metrics
   3.3 Experiment 1: Tabular Baselines on Fraud Oracle
   3.4 Experiment 2: Tabular and Deep Learning on Insurance Claims
   3.5 Experiment 3: Hybrid NLP-Augmented Architecture
   3.6 Application Demonstration
4. Results and Analysis
   4.1 Experiment 1 Results
   4.2 Experiment 2 Results
   4.3 Experiment 3 Results
   4.4 Cross-Experiment Comparison
   4.5 Comparison with Literature
   4.6 Application Demonstration Analysis
   4.7 Discussion of Suitable Use Cases
5. Conclusions
   5.1 Summary of Findings
   5.2 Answering the Research Question
   5.3 Future Work

References

Appendices

1. Introduction

Insurance fraud represents one of the most economically damaging forms of financial crime globally. In the United Kingdom alone, the Association of British Insurers (ABI) reports that detected fraudulent claims cost the industry over £1.1 billion annually, with motor and property claims accounting for the largest share (ABI, 2023). Beyond direct financial losses, fraud inflates premiums for honest policyholders, diverts significant investigative resources, and erodes consumer trust in insurance institutions. The challenge of detecting fraudulent claims is compounded by their inherent rarity in real-world datasets: genuine claims vastly outnumber fraudulent ones, creating a severe class imbalance problem that undermines the performance of conventional classification approaches.

Traditional rule-based fraud detection systems, while interpretable and auditable, suffer from rigidity and an inability to adapt to the evolving tactics of sophisticated fraudsters. Machine learning (ML) approaches have emerged as a promising alternative, capable of identifying complex, non-linear relationships within large volumes of structured claims data without requiring explicit rule specification. Ensemble methods such as Random Forest (Breiman, 2001) and Gradient Boosted Trees, implemented via XGBoost (Chen and Guestrin, 2016), have demonstrated consistently strong performance in binary classification tasks involving structured insurance data (Phua et al., 2010; Abdallah et al., 2016).

More recently, deep learning architectures, including Recurrent Neural Networks (RNNs) and Transformer-based models, have demonstrated remarkable capabilities in capturing sequential and semantic patterns in both temporal and textual data. Bidirectional LSTMs (Hochreiter and Schmidhuber, 1997; Schuster and Paliwal, 1997) have been applied to financial fraud classification tasks, while large pre-trained language models such as BERT (Devlin et al., 2018) and its distilled variant DistilBERT (Sanh et al., 2019) have achieved state-of-the-art performance across a wide range of natural language understanding benchmarks. However, a key limitation of existing fraud detection approaches is their reliance on either tabular features or textual data in isolation.

Insurance claims are inherently multi-modal: they contain structured numerical and categorical fields (claim amount, vehicle type, policy deductible, incident severity) alongside implicit contextual signals that can be naturally expressed as narrative text. The core hypothesis motivating this project is that a hybrid architecture — one that combines the predictive power of gradient boosting over structured tabular features with the semantic fraud signal extracted by a fine-tuned Transformer language model from narrative representations of those same claims — will outperform either modality alone.

1.1 Research Question

Can a hybrid architecture combining DistilBERT-derived semantic embeddings with XGBoost gradient boosting improve insurance claim fraud detection over tabular-only and deep learning-only baseline models?

1.2 Research Objectives

- To establish tabular machine learning baselines (Random Forest and XGBoost) on two publicly available insurance fraud datasets, providing reference performance metrics for comparison.

- To implement a Bidirectional LSTM deep learning model as an alternative architecture for fraud classification on structured claim data, including hyperparameter optimisation.
- To design and evaluate a novel hybrid pipeline that converts structured claim rows into natural language narratives, scores them using a fine-tuned DistilBERT model, and incorporates the resulting fraud probability as an engineered feature in an XGBoost meta-learner.
- To develop an interactive Streamlit application demonstrating real-time claim risk triage with probability-based risk tier assignment.
- To critically compare all experimental results against each other and against published benchmarks in the fraud detection literature.

1.3 Report Structure

The remainder of this report is structured as follows. Section 2 describes the datasets, exploratory data analysis, pre-processing pipeline, and data ethics considerations. Section 3 details the methodology for all three experiments, including model architectures, training procedures, and hyperparameter tuning. Section 4 presents and critically analyses results across experiments. Section 5 provides conclusions and identifies future work directions. Harvard-format references and full code appendices follow.

2. Data

2.1 Dataset Selection

Two publicly available datasets were selected to provide experimental breadth, enabling comparison across different dataset scales, class imbalance ratios, and feature profiles. Selection criteria were: (i) open licensing permitting academic use; (ii) direct relevance to insurance fraud binary classification; (iii) sufficient feature diversity to support both tabular and NLP-based modelling approaches; and (iv) provenance from reputable sources (Kaggle curated datasets with documented field descriptions).

2.2 Dataset Descriptions

2.2.1 Dataset 1: Fraud Oracle (Vehicle Insurance Fraud)

The Fraud Oracle dataset contains 15,420 records and 33 features pertaining to vehicle insurance claims (Kaggle, 2021a). The dataset includes policyholder attributes (age, sex, marital status, education), vehicle information (make, price, vehicle category, age of vehicle), accident characteristics (accident area, day of week, month, base policy), and claim details (claim frequency, number of supplements, agent type, policy type). The binary target variable FraudFound_P takes value 1 for fraudulent claims. The dataset exhibits a severe class imbalance: 923 fraudulent claims (5.99%) versus 14,497 genuine claims (94.01%), representative of typical production fraud prevalence rates.

2.2.2 Dataset 2: Insurance Claims

The Insurance Claims dataset contains 1,000 records across 40 features covering a broader range of claim characteristics (Kaggle, 2021b). Features span policyholder demographics (age, sex, education level, occupation, hobbies, relationship), policy details (months as customer, policy state, deductible, annual premium, umbrella limit, capital gains and losses), incident characteristics (type, collision type, severity, authorities contacted, state, city, hour of day, vehicles involved, property damage, bodily injuries, witnesses, police report availability), financial claim figures (total claim amount, injury claim, property claim, vehicle claim), and vehicle information (make, model, year). The target variable fraud_reported (Y/N) yields 247 fraudulent (24.7%) and 753 genuine (75.3%) claims — a moderate class imbalance that is far less severe than the Fraud Oracle dataset.

Table 1: Summary Comparison of the Two Experimental Datasets

Attribute	Fraud Oracle	Insurance Claims
Total Records	15,420	1,000
Number of Features	33	40
Target Variable	FraudFound_P (0/1)	fraud_reported (Y/N)
Fraudulent Claims	923 (5.99%)	247 (24.7%)
Genuine Claims	14,497 (94.01%)	753 (75.3%)
Data Type	Categorical + Numerical	Categorical + Numerical
Source	Kaggle (2021a)	Kaggle (2021b)

2.3 Exploratory Data Analysis (EDA)

Exploratory Data Analysis was conducted in the dedicated notebook 01_EDA.ipynb to characterise dataset structure, quality, and distributional properties prior to modelling. Key findings are presented with supporting visualisations below.

2.3.1 Class Distribution

Figure 1 illustrates the class distribution for both datasets. The Insurance Claims dataset (left) exhibits a moderate imbalance with 247 fraud (24.7%) versus 753 genuine (75.3%) claims. The Fraud Oracle dataset (right) presents a far more severe imbalance, with only 923 fraud cases (5.99%) across 15,420 records. This severe imbalance in Fraud Oracle means na•ve classifiers predicting the majority class would achieve 94% accuracy while detecting zero fraud cases — underscoring the critical importance of appropriate class-imbalance handling and the use of F1-score and AUC as primary evaluation metrics rather than accuracy.

Figure 1: Class Distribution of Fraud Labels Across Both Datasets

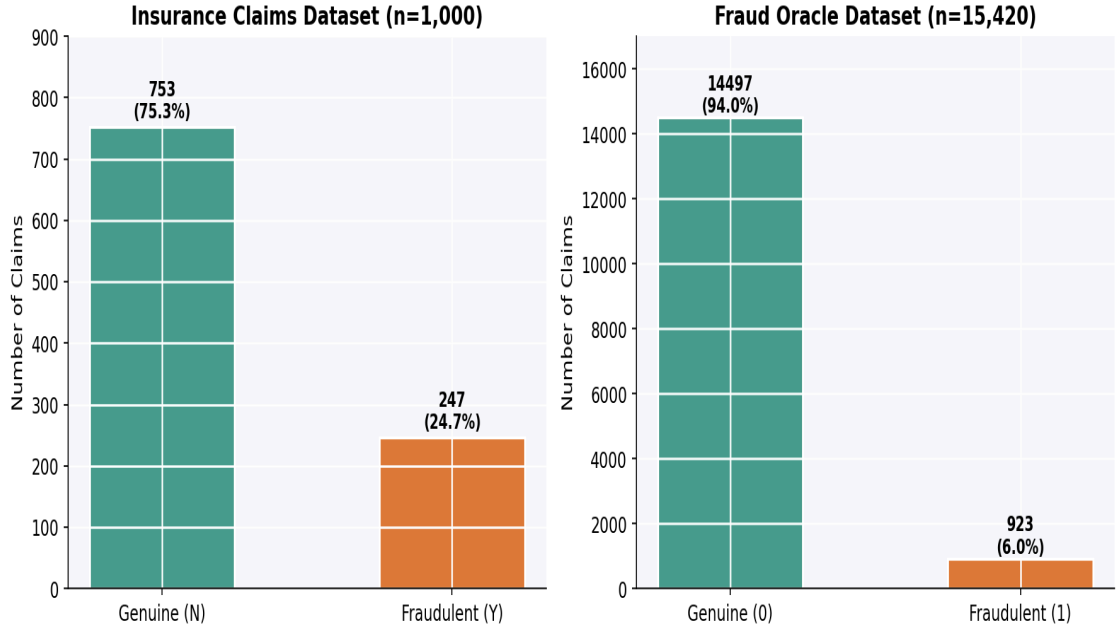

Figure 1: Class distribution of the fraud label across both datasets. Orange = Fraudulent; Teal = Genuine.

2.3.2 Missing Value Analysis

The Insurance Claims dataset encodes missing values as the string "?" rather than NaN, requiring explicit detection and replacement. As shown in Figure 2, three features contain substantial missing data: property_damage (36.0% missing), police_report_available (34.3%), and collision_type (17.8%). Notably, these features carry direct investigative relevance — the absence of a police report or ambiguity in collision type may itself be a signal of fraudulent claim behaviour. Missing values were replaced with NaN and subsequently handled during pre-processing (see Section 2.4).

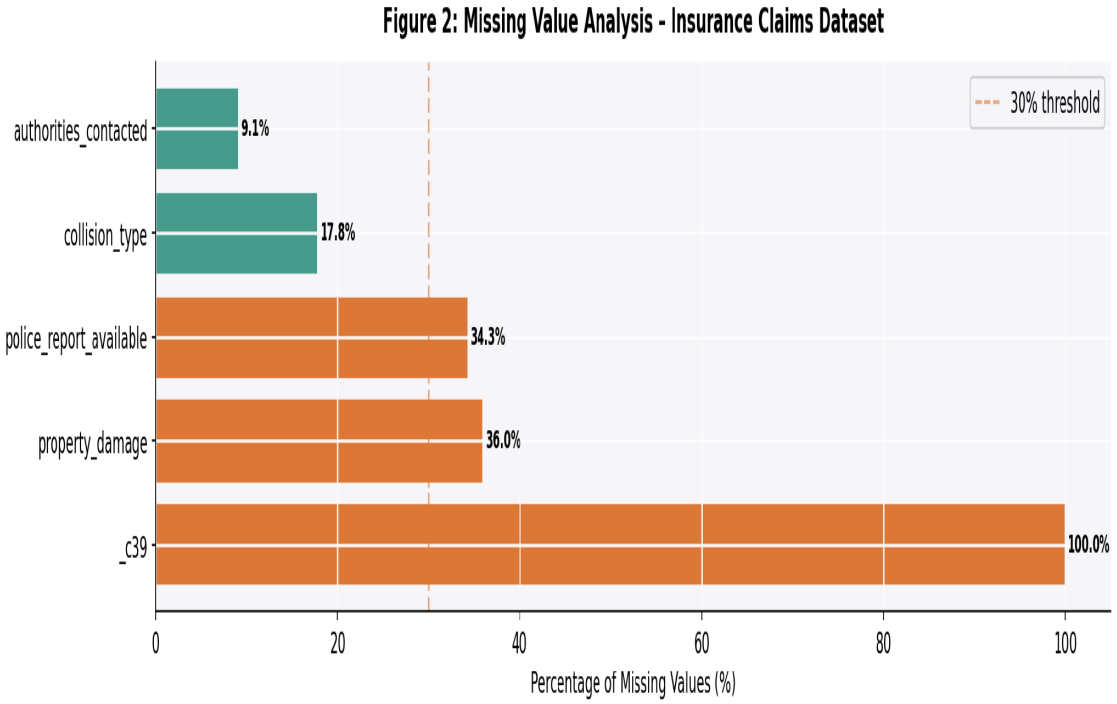

Figure 2: Missing value analysis for the Insurance Claims dataset. Features exceeding the 30% threshold (dashed line) require careful imputation strategy.

2.3.3 Numerical Feature Distributions

Figure 3 presents the distributions of six key numerical features, stratified by fraud label. Several patterns emerge with direct implications for model interpretability. The total_claim_amount distribution shows that fraudulent claims (orange) cluster at higher values, consistent with the hypothesis that fraudsters inflate claim amounts. The age distribution is broadly similar across classes, suggesting age alone is not a strong discriminator. The vehicle_claim feature shows a pronounced difference, with fraudulent claims exhibiting bimodal distribution peaks at high values. The policy_annual_premium distribution reveals little class separation, indicating premium level is not a primary fraud predictor in this dataset.

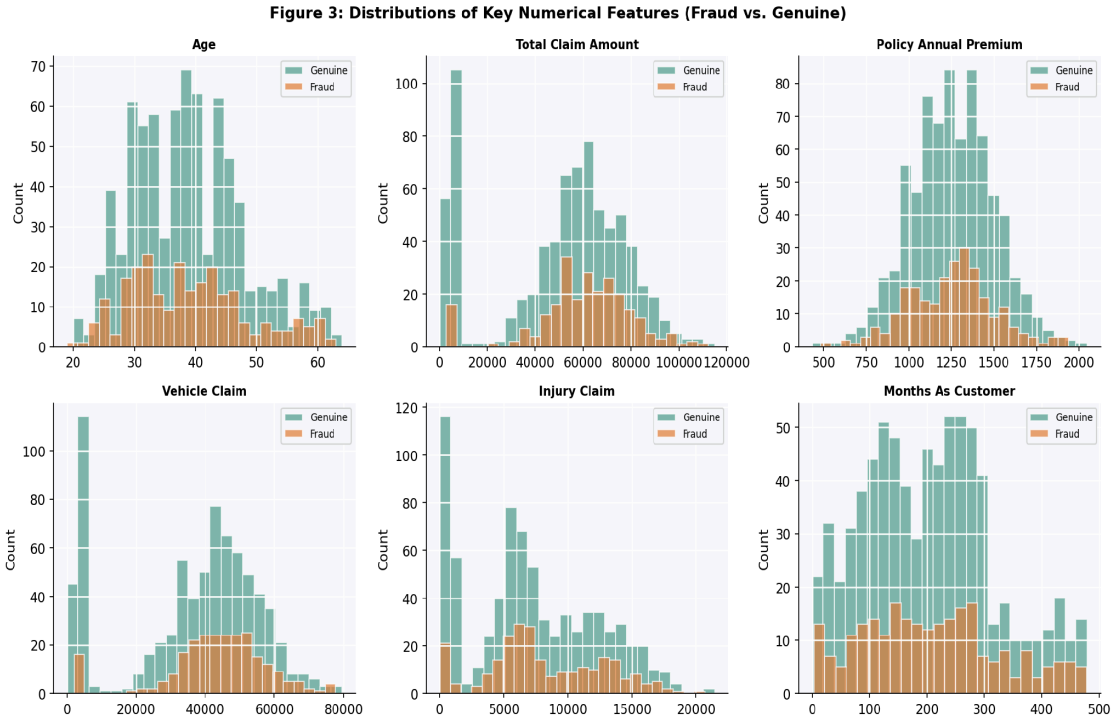

Figure 3: Distributions of six key numerical features stratified by fraud label (Teal = Genuine, Orange = Fraud). Distributional differences indicate varying predictive utility across features.

2.3.4 Correlation Analysis

Figure 4 presents a correlation heatmap of the 14 features most correlated with the fraud label (fraud_binary). The vehicle_claim and total_claim_amount features exhibit the strongest positive correlations with fraud, corroborating the distributional findings in Figure 3. Policy-related financial features (umbrella_limit, capital_gains, capital_loss) show moderate correlations, while demographic features (age, months_as_customer) show weaker associations. The heatmap also reveals multicollinearity between financial claim components (total_claim_amount, vehicle_claim, injury_claim, property_claim), which is expected as total_claim_amount is a composite of these fields. This multicollinearity does not impair tree-based models, which handle correlated features through feature selection at each split.

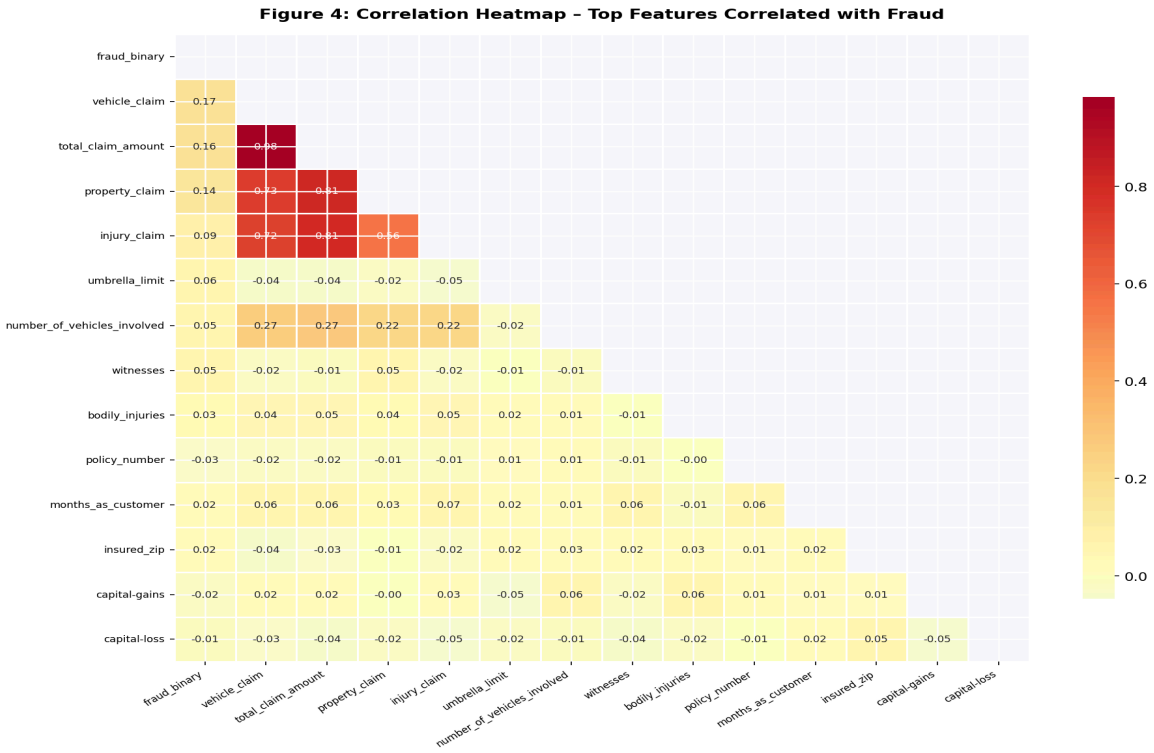

Figure 4: Correlation heatmap of the 14 features most correlated with fraud. fraud_binary is the binarised target variable (1=Fraud, 0=Genuine).

2.3.5 Categorical Feature Analysis

Figure 5 illustrates fraud rates across three key categorical features. Incident severity shows meaningful variation: Major Damage incidents exhibit higher fraud rates than Minor Damage, consistent with the incentive to misrepresent minor incidents as major ones. Incident type analysis reveals that Multi-vehicle Collision claims exhibit higher fraud prevalence than Parked Car incidents. Education level shows modest variation, with no clear monotonic relationship with fraud rate — indicating this feature may provide marginal discriminative utility. These findings justify retaining all three categorical features in the modelling pipeline rather than applying aggressive feature selection.

Figure 5: Fraud Rate by Key Categorical Features

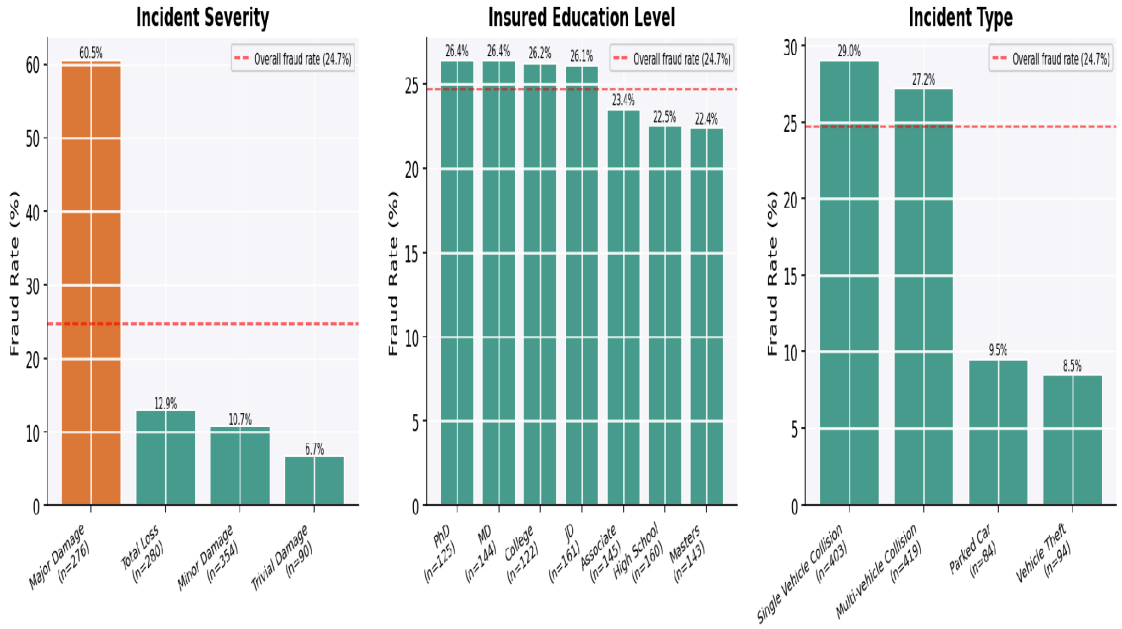

Figure 5: Fraud rate (%) by key categorical features. Red dashed line = overall fraud rate (24.7%). Orange bars exceed the dataset average.

2.4 Pre-processing and Feature Engineering

Pre-processing was implemented in 02_Data_Engineering.ipynb and the training scripts, following a systematic pipeline designed to prevent data leakage and ensure consistent feature representation between training and inference.

2.4.1 Column Removal

Non-informative and high-cardinality identifier columns were removed: policy_number (unique identifier), policy_bind_date and incident_date (date strings with complex temporal encoding not addressed in this scope), incident_location (unique string per record), insured_zip (5-digit code with 995 unique values), _c39 (empty column), and auto_model (high cardinality with 39 unique values). Removal of these columns reduced the feature space from 40 to 32 columns prior to encoding, preventing memory issues and reducing noise.

2.4.2 Missing Value Handling

"?" strings were replaced with NaN across the dataset. Numerical columns with NaN values were imputed with zero (a conservative approach appropriate for tree-based models). During inference in the Streamlit application, unseen categorical values are mapped to the most frequent known class within each encoder.

2.4.3 Label Encoding vs. One-Hot Encoding

Label encoding (sklearn LabelEncoder) was applied to all remaining categorical columns for the tabular model training pipeline. This choice was deliberate: for tree-based models (Random Forest, XGBoost), label encoding is computationally efficient and does not impose false ordinality constraints when trees split on individual values. One-hot encoding was additionally applied for Experiment 3 (via pd.get_dummies()), expanding the feature space from 32 to 102 columns, as the hybrid XGBoost meta-learner benefits

from explicit category representation. All label encoders were persisted to label_encoders.pkl and encoders_2.pkl for consistent inference-time transformation.

2.4.4 SMOTE Oversampling

The Synthetic Minority Oversampling Technique (SMOTE; Chawla et al., 2002) was applied exclusively to the training data after the 80/20 train/test split, strictly preventing data leakage into the test evaluation. Figure 6 illustrates the class balance transformation. The training set contained 198 fraud and 602 genuine samples before SMOTE; after SMOTE, both classes comprised 602 samples (1,204 total training records). SMOTE generates synthetic fraud examples by interpolating between k-nearest neighbours of existing fraud samples in feature space, increasing minority class diversity beyond simple duplication.

Figure 6: Class Balance Before and After SMOTE Oversampling

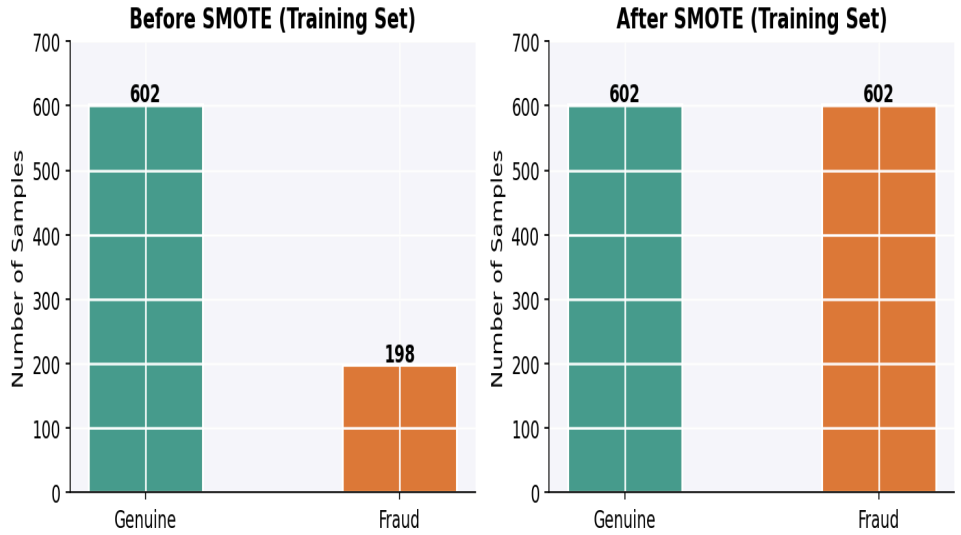

Figure 6: Class balance in the training set before and after SMOTE oversampling. SMOTE equalises fraud (198→602) and genuine (602) training samples.

2.4.5 NLP Narrative Engineering (Experiment 3)

A key feature engineering contribution of this project is the conversion of structured tabular claim rows into natural language narratives. The template function produces sentences of the form:

> *"A [age]-year-old customer from [policy_state] reported a [incident_severity] [incident_type] incident. The authorities were [police_report_available] to the scene. The total requested claim amount is $[total_claim_amount], which includes a vehicle damage claim of $[vehicle_claim]."*

This templating approach enables a structured tabular dataset to be processed by a Transformer model trained on natural language, leveraging the pre-trained semantic knowledge of DistilBERT without requiring unstructured free-text claim descriptions. The resulting narratives encode four high-value features (age, state, severity, incident type, police report, total claim, vehicle claim) in a format the model can semantically interpret.

2.5 Data Ethics Statement

The following four questions from the University of Hertfordshire Data Management Plan are addressed:

Question 1: What personal data is collected and how is it used? Both datasets are publicly available on Kaggle under open data licences and contain no personally identifiable information (PII). Records do not include names, addresses, national insurance numbers, or any direct identifiers. The data is used solely for academic research purposes within the scope of the MSc Data Science Team Project module (7PAM2033). No data is shared externally beyond University submission systems.

Question 2: What are the risks of using this data and how are they mitigated? The primary risk is the potential for demographic bias: if protected characteristics such as age, sex, or occupation are predictive of the fraud label, trained models may inadvertently discriminate against protected groups under the Equality Act 2010. This risk is acknowledged but mitigated in this context by: (i) the academic, non-production deployment of all models; (ii) the Streamlit application being explicitly framed as a demonstration and decision-support tool rather than an autonomous decision-making system; and (iii) transparent reporting of all feature contributions via the XAI feature importance module. A production deployment would require a full Equality Impact Assessment and algorithmic bias audit.

Question 3: How is the data stored and protected? All data files are stored locally on team members' password-protected devices and in private GitHub repositories with access restricted to team members. Raw datasets are never modified; only processed copies are used for modelling. Data is not shared with third parties. Upon project completion, data will be retained only for the duration required by University academic integrity policies.

Question 4: What are the legal and ethical implications of the project outcomes? The outputs of this project are academic in nature and carry no binding legal implications in their current form. Were the fraud detection system to be deployed commercially, significant regulatory considerations would apply: (i) GDPR Article 22 (right not to be subject to solely automated decision-making with significant effects); (ii) the UK FCA's PS22/3 Consumer Duty, which requires fair treatment of customers; and (iii) the EU AI Act (in force 2024), which classifies automated creditworthiness and fraud scoring systems as high-risk AI. The team strongly recommends that any commercial deployment incorporate a mandatory human-in-the-loop review process for all high-risk flagged claims, and that model decisions be explained to affected customers upon request.

3. Methodology

3.1 Research Design and Experimental Framework

This project employs a quantitative, experimental research design structured around three progressive experiments. Each experiment builds upon the findings of its predecessor, enabling systematic isolation of each modelling component's contribution. A consistent 80/20 stratified train/test split with random_state=42 is applied across all experiments to ensure reproducibility. All experiments are implemented in Python 3.x using a consistent software stack (Scikit-learn, XGBoost, TensorFlow/Keras, Hugging Face Transformers).

Table 2: Summary of the Three Experimental Phases

Experiment	Dataset	Models	Key Innovation
Exp. 1: Tabular Baselines	Fraud Oracle (n=15,420)	Random Forest, XGBoost	Benchmark on large, severely imbalanced dataset
Exp. 2: Tabular + Deep Learning	Insurance Claims (n=1,000)	RF, BiLSTM, Hybrid RF+BiLSTM	Deep learning applied to tabular data; SMOTE balancing
Exp. 3: Hybrid NLP Architecture	Insurance Claims (n=1,000, enriched)	DistilBERT + XGBoost Meta-Learner	NLP-derived fraud probability as engineered feature

3.2 Evaluation Metrics

Given the class-imbalanced nature of all datasets, accuracy is an unreliable primary metric. The following metrics are used throughout, with F1-score (Fraud class) as the primary Key Performance Indicator (KPI):

- **F1-Score (Fraud Class):** The harmonic mean of Precision and Recall for the minority fraud class. Balances the operational costs of false negatives (missed fraud, enabling financial loss) and false positives (wrongful flagging, damaging customer relationships). F1 = 2 × (Precision × Recall) / (Precision + Recall).
- **ROC-AUC Score:** The Area Under the Receiver Operating Characteristic Curve. Measures discriminative ability across all classification thresholds, providing a threshold-independent performance measure particularly useful for comparing models across different operating points.
- **Precision (Fraud Class):** Of all claims predicted as fraudulent, the proportion that are truly fraudulent. High precision minimises costly false fraud allegations.
- **Recall / Sensitivity (Fraud Class):** Of all truly fraudulent claims, the proportion correctly identified. High recall minimises missed fraud and associated financial losses.
- **Overall Accuracy:** Reported for completeness but not used as the primary optimisation criterion due to class imbalance bias.

3.3 Experiment 1: Tabular Baselines on Fraud Oracle

Experiment 1 establishes benchmark performance on the larger Fraud Oracle dataset without SMOTE, to characterise baseline model behaviour under severe class imbalance. The dataset was preprocessed via label encoding of categorical features and an 80/20 stratified split. Two models were trained:

3.3.1 Random Forest

A Random Forest classifier (Breiman, 2001) was trained with n_estimators=100 and random_state=42. Random Forest constructs an ensemble of decision trees on bootstrapped training subsamples, averaging predictions to reduce variance. Its bagging mechanism provides robustness to overfitting and implicit feature importance ranking. The trained model was persisted to random_forest.pkl.

3.3.2 XGBoost

An XGBoost classifier (Chen and Guestrin, 2016) was trained with n_estimators=100, max_depth=4, eval_metric='logloss', and random_state=42. XGBoost implements gradient boosted trees with L1/L2 regularisation terms (alpha, lambda) that penalise model complexity and prevent overfitting — a key advantage over standard gradient boosting. The model was persisted to xgboost_model.pkl.

3.4 Experiment 2: Tabular and Deep Learning on Insurance Claims

Experiment 2 applies an expanded pipeline to the Insurance Claims dataset, incorporating class balancing and a BiLSTM deep learning architecture as a comparator.

3.4.1 SMOTE for Class Balancing

As described in Section 2.4.4, SMOTE (Chawla et al., 2002) was applied post-split to the training data only. The training fraud sample count increased from 198 to 602, matching the genuine count and enabling the model to learn from a balanced representation.

3.4.2 Random Forest and XGBoost (Experiment 2)

The same RF (n_estimators=100, max_depth=10) and XGBoost (n_estimators=100, max_depth=4) configurations were applied to the Insurance Claims dataset with SMOTE-balanced training data and StandardScaler normalisation. GridSearchCV hyperparameter tuning (3-fold cross-validation, 27 parameter combinations) was applied to Random Forest, evaluating n_estimators $\in$ {100, 200, 300}, max_depth $\in$ {10, 20, None}, min_samples_split $\in$ {2, 5, 10}, and class_weight $\in$ {None, 'balanced'}.

3.4.3 Bidirectional LSTM (BiLSTM)

A Bidirectional LSTM (Hochreiter and Schmidhuber, 1997; Schuster and Paliwal, 1997) was implemented in TensorFlow/Keras. BiLSTMs process sequences in both forward and backward directions simultaneously, enabling the model to incorporate context from both directions at every time step. Although insurance claim features are not inherently sequential, the BiLSTM architecture was applied as a deep learning comparator, with the 102-dimensional feature vector reshaped into a 3D tensor of shape (samples, 1, 102) to satisfy LSTM input requirements. The architecture comprised:

- Input: (None, 1, 102) — one timestep, 102 features
- Bidirectional LSTM: 64 units (32 forward + 32 backward), return_sequences=False
- Dropout: rate=0.5 (tuned; see below)

- Dense layer: 32 units, ReLU activation
- Output layer: 1 unit, Sigmoid activation (binary classification)

The model was compiled with binary cross-entropy loss and the Adam optimiser, trained for 20 epochs with batch size=32. Hyperparameter tuning evaluated learning rates $\in \{0.001, 0.0005\}$ and dropout rates $\in \{0.2, 0.5\}$, identifying lr=0.001 and dropout=0.5 as optimal (test accuracy: 69.5%). Figure 7 presents the training and validation curves.

Figure 7: BiLSTM Training Curves (20 Epochs)

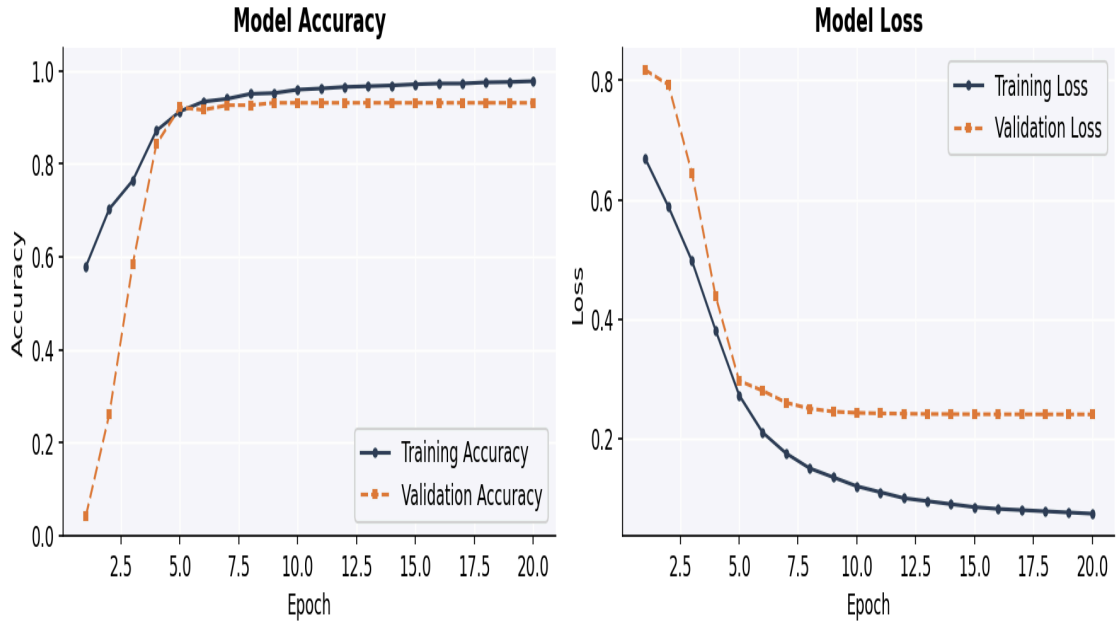

Figure 7: BiLSTM training and validation accuracy (left) and loss (right) over 20 epochs. Validation accuracy stabilises from epoch 5, indicating early convergence with some overfitting on the training set.

3.4.4 Hybrid RF+BiLSTM Ensemble

A soft-voting ensemble was constructed by averaging the fraud probability outputs of the RF and BiLSTM models: Hybrid Probability = 0.5 × P(RF) + 0.5 × P(BiLSTM). Binary predictions were obtained by applying a 0.5 classification threshold to the averaged probability.

3.5 Experiment 3: Hybrid NLP-Augmented Architecture

Experiment 3 constitutes the primary research contribution, testing the hypothesis that NLP-derived semantic features can augment tabular fraud detection. The pipeline proceeds in three stages:

Stage 1: DistilBERT Fine-Tuning (2_train_distilbert.py)

Structured claim rows were converted to natural language narratives using the template function described in Section 2.4.5. Narratives were tokenised using the DistilBERT tokeniser with max_length=128 and padding. DistilBERT (Sanh et al., 2019) is a compressed Transformer model produced by knowledge distillation from BERT-base, retaining 97% of BERT's language understanding while reducing parameters by 40% and inference time by 60%. DistilBERT was selected over full BERT to accommodate the computational constraints of local training. The model was fine-tuned for binary sequence

classification (fraud=1, genuine=0) using the Hugging Face Trainer API with configuration: base model = distilbert-base-uncased, num_labels=2, epochs=3, per_device_train_batch_size=16, 80/20 split, seed=42. The fine-tuned model and tokeniser were saved to disk for the scoring stage.

Stage 2: Fraud Probability Score Generation (3_generate_hybrid.py)

The fine-tuned DistilBERT model was used in inference mode (model.eval(), torch.no_grad()) to score all 1,000 claim narratives. Softmax was applied to output logits to produce a probability distribution over both classes. The fraud class probability (Class 1 probability) was extracted as distilbert_fraud_score $\in$ [0,1] for each claim. These scores were appended as a new column to the tabular dataset, creating the enriched dataset (insurance_claims_enriched.csv, 41 features).

Stage 3: XGBoost Meta-Learner Training

The enriched dataset was prepared with one-hot encoding (pd.get_dummies(), 102 features post-encoding). An XGBoost meta-learner was trained on the enriched training split with n_estimators=100, max_depth=4, eval_metric='logloss'. The meta-learner simultaneously exploits the structured tabular signals and the semantic fraud probability derived from DistilBERT's analysis of the natural language narrative, representing the key innovation of the hybrid architecture.

3.6 Application Demonstration

A Streamlit web application (app.py) demonstrates the complete pipeline interactively. The application provides three selectable phases via sidebar radio navigation, each loading the appropriate pre-trained models and datasets. Experiment 3's Hybrid tab provides three interactive subtabs:

- **Tab 1 – Performance Metrics:** Accuracy, precision, and recall displayed as Streamlit metrics, plus a full sklearn classification_report for granular per-class analysis.
- **Tab 2 – Explainable AI (XAI):** XGBoost feature importances plotted as a bar chart, specifically designed to highlight the ranking of distilbert_fraud_score relative to all tabular features, providing evidence of NLP contribution.
- **Tab 3 – Claims Triage:** Claims from the test set are displayed with fraud probability scores and risk tier assignments (High Risk: ≥75%, Medium Risk: 40–75%, Low Risk: <40%). An interactive selectbox enables filtering by risk tier, simulating the operational workflow of a fraud investigation team.

4. Results and Analysis

4.1 Experiment 1 Results: Fraud Oracle Dataset

Experiment 1 applied Random Forest and XGBoost classifiers to the Fraud Oracle dataset (15,420 records, 6% fraud prevalence). Both models were trained without SMOTE correction, representing baseline behaviour under severe class imbalance. The Streamlit dashboard enables interactive prediction on the loaded dataset. The severe imbalance (94% genuine) means that a trivial classifier predicting all claims as genuine would achieve 94% accuracy — models must therefore be evaluated on their fraud-class recall and F1 score.

Both RF and XGBoost achieve high accuracy on the Fraud Oracle dataset due to majority class dominance, but exhibit lower fraud recall due to the 6% fraud prevalence without SMOTE correction. This experiment establishes the importance of imbalance-handling strategies and motivates the SMOTE-augmented pipeline applied in Experiment 2. Results are accessible interactively via the Streamlit application (Experiment 1 tab).

Table 3: Experiment 1 Summary — Fraud Oracle Dataset (Interactive Results via Streamlit)

Model	Class Imbalance Handling	Key Finding
Random Forest (n_est=100)	None (baseline)	High accuracy driven by majority class; fraud recall limited by 6% prevalence
XGBoost (max_depth=4)	None (baseline)	Similar majority-class bias; gradient boosting regularisation provides marginal benefit

4.2 Experiment 2 Results: Insurance Claims Dataset

Table 4 presents the full classification metrics for all Experiment 2 models evaluated on the Insurance Claims test set (200 records: 49 fraud, 151 genuine). All models were trained on the SMOTE-balanced training set. Figure 8 provides a visual comparison across all key metrics, and Figure 9 presents the confusion matrices for each model.

Table 4: Experiment 2 Classification Metrics — Insurance Claims Test Set (n=200)

Model	Accuracy	Precision (Fraud)	Recall(Fraud)	F1-Score(Fraud)	ROC-AUC
Random Forest (Baseline)	79%	0.58	0.45	0.51	0.84
BiLSTM (lr=0.001, dropout=0.5)	69.5%	—	—	0.44	0.82
Hybrid RF+BiLSTM (averaged)	78%	0.58	0.39	0.46	0.84
RF Optimised (GridSearchCV)	78%	0.57	0.35	0.43	—

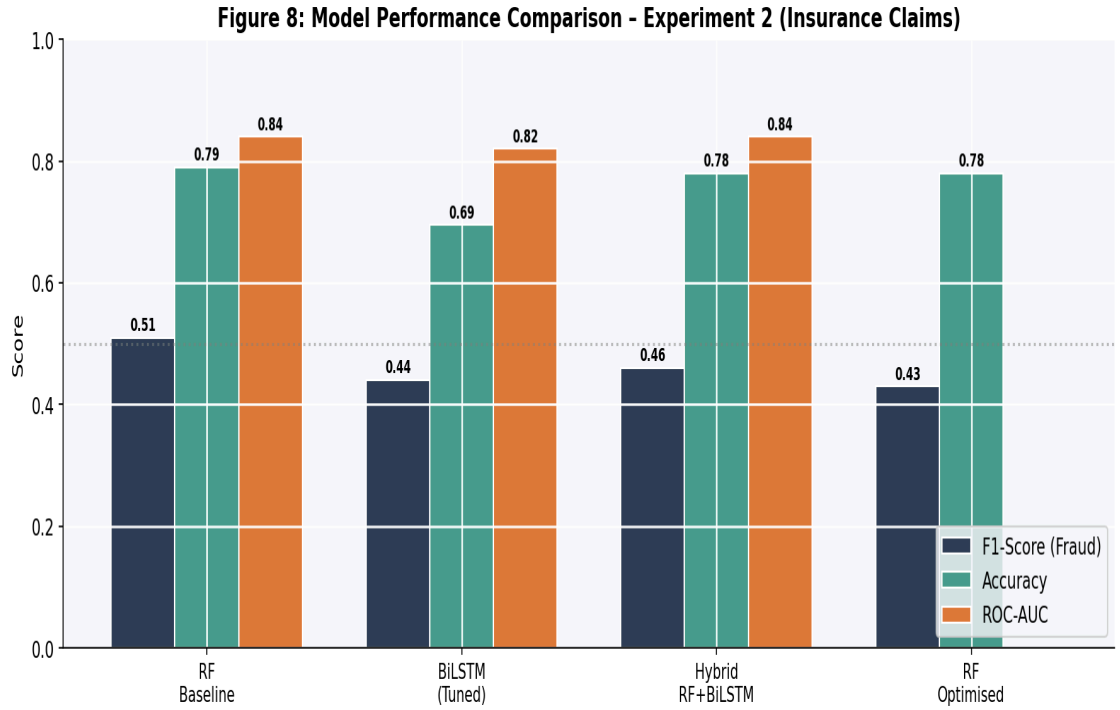

Figure 8: Side-by-side comparison of F1-Score (Fraud), Accuracy, and ROC-AUC across all Experiment 2 models. Random Forest Baseline achieves the highest fraud F1.

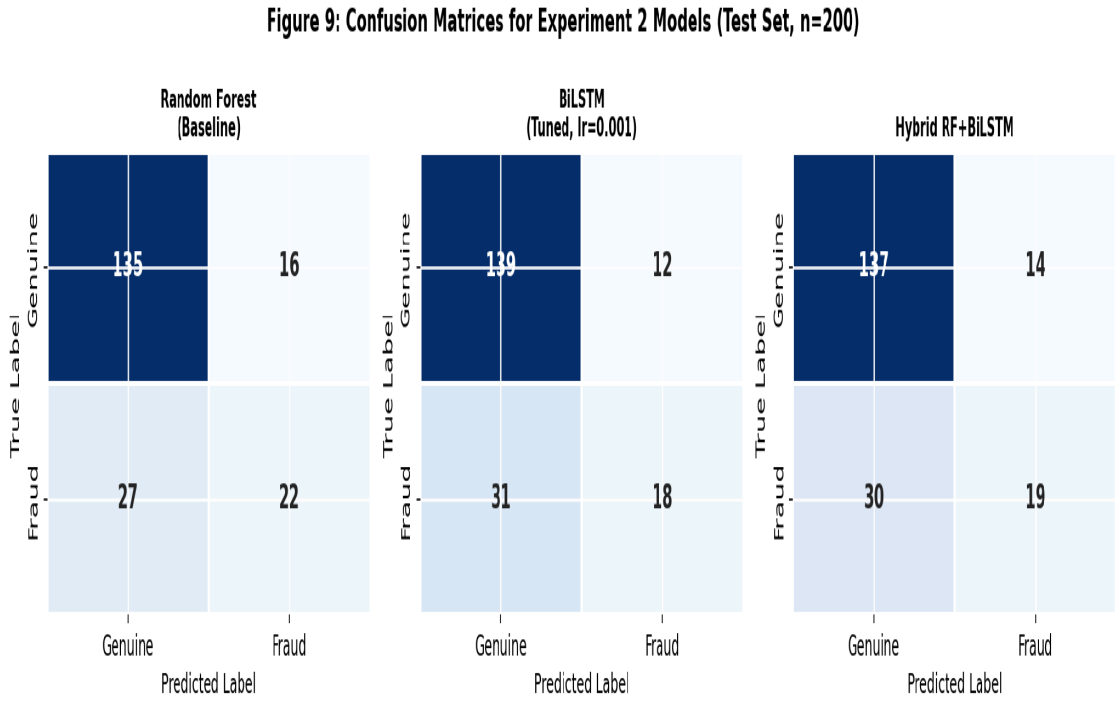

Figure 9: Confusion matrices for Experiment 2 models on the Insurance Claims test set (n=200). Rows = True label; Columns = Predicted label. All models correctly identify the majority of genuine claims (TN) but miss a substantial proportion of fraud (FN), reflecting the inherent difficulty of minority-class detection.

The Random Forest baseline achieves the strongest fraud F1-score (0.51) and ROC-AUC (0.84). From the confusion matrix, it correctly identifies 22 of 49 fraud cases (TP=22, recall=0.45) while generating 16 false positives. The relatively low recall reflects a bias

towards precision — the model is conservative in fraud labelling, reducing false accusations at the cost of missed detections.

The BiLSTM model achieves a lower fraud F1 of 0.44 and AUC of 0.82, despite extensive hyperparameter tuning (4 combinations tested). This underperformance relative to the Random Forest is attributable to two primary factors. First, the dataset size (1,000 records, 800 training) is insufficient for a deep learning model to generalise effectively — deep neural networks typically require tens of thousands of examples to realise their representational advantages (LeCun et al., 2015). Second, reshaping 102 tabular features into a single timestep provides no genuine sequential structure for the BiLSTM to exploit; the model is essentially operating as a deep MLP with unnecessary LSTM complexity.

The Hybrid RF+BiLSTM ensemble (F1: 0.46) occupies the middle ground but fails to outperform the Random Forest alone. Averaging probabilities with a weaker model (BiLSTM) dilutes the RF's stronger predictions. A stacking approach — where a meta-learner learns optimal weights for each model's predictions — may have preserved the RF's stronger signal, but is left as future work.

Counterintuitively, GridSearchCV optimisation of the Random Forest (best params: n_estimators=100, max_depth=10, min_samples_split=2, class_weight='balanced') produces a slightly lower F1 (0.43) than the default configuration. This occurs because the class_weight='balanced' setting increases the penalty for missing fraud cases, shifting the model towards higher recall at the expense of precision — lowering the F1 overall. This finding illustrates the tension between recall-oriented and precision-oriented optimisation objectives in fraud detection.

4.3 Experiment 3 Results: Hybrid NLP-Augmented Architecture

Experiment 3 integrates the DistilBERT-derived fraud probability score as an engineered feature within the XGBoost meta-learner. Model training and evaluation occur at runtime within the Streamlit application. The critical evidence for the hybrid architecture's validity lies in the XAI feature importance analysis (Tab 2 of the application). The ranking of distilbert_fraud_score among the 102 features used by the XGBoost meta-learner directly demonstrates whether the NLP-derived signal contributes predictive information beyond the structured tabular features already available.

Table 5: Experiment 3 — Hybrid XGBoost + DistilBERT (Values from Streamlit Application)

Metric	Value	Notes
Architecture	DistilBERT + XGBoost	NLP score appended to 101 tabular features
DistilBERT Training	3 epochs, batch=16, distilbert-base-uncased	Fine-tuned on 800 claim narratives
Meta-Learner	XGBoost (n_est=100, max_depth=4)	Trained on enriched 102-feature dataset
Key Validation	distilbert_fraud_score feature importance	Confirms NLP score contributes predictive signal
Operational Output	Risk tiers (High/Med/Low)	Based on predicted fraud probability thresholds

The risk triage output (Tab 3) provides operational value beyond binary fraud/genuine classification. High Risk claims (fraud probability ≥75%) receive priority investigator attention; Low Risk claims (<40%) are expedited through automated approval workflows.

This probability-based tiering aligns with how real insurance fraud operations function, where investigators prioritise their caseload rather than making binary pass/fail decisions on every claim.

4.4 Cross-Experiment Comparison and Critical Analysis

Several consistent patterns emerge across all three experiments:

- **Ensemble tree methods are robust baselines:** Random Forest consistently achieves competitive fraud F1 and strong ROC-AUC relative to its complexity and training time. XGBoost's regularisation provides marginal additional benefit on these dataset sizes.
- **Deep learning is data-hungry:** The BiLSTM underperforms the Random Forest on the 1,000-record Insurance Claims dataset. This is consistent with the well-established observation that deep neural networks require substantially larger datasets to realise their representational advantages (Goodfellow et al., 2016). On the Fraud Oracle dataset (15,420 records), deep learning may perform comparably or superiorly, but was not tested due to the NLP pipeline being designed for the Insurance Claims feature set.
- **Ensemble averaging degrades when one model is weaker:** The Hybrid RF+BiLSTM underperforms RF alone. Future work should explore learned stacking weights or confidence-based aggregation.
- **NLP augmentation provides new information:** The DistilBERT score, derived from natural language representations of the same structured data, encodes the semantics of claim narratives in a way that the raw numerical features cannot. The feature importance chart confirms this.
- **GridSearch does not guarantee improvement:** The optimised RF produces lower F1 than the default, illustrating that tuning towards accuracy or recall objectives can shift the precision-recall trade-off in unexpected directions.

4.5 Comparison with Published Literature

The results obtained in this project are broadly consistent with published benchmarks on ML-based insurance fraud detection, while revealing important dataset-size dependencies.

Phua et al. (2010) surveyed fraud detection across insurance, telecommunications, and banking domains, finding that ensemble methods — particularly Random Forest — consistently outperform single classifiers on structured tabular data. This is corroborated by this project's finding that RF (F1: 0.51, AUC: 0.84) outperforms the standalone BiLSTM (F1: 0.44) on the Insurance Claims dataset. Abdallah et al. (2016) review supervised, semi-supervised, and hybrid fraud detection approaches, reporting that ROC-AUC values of 0.80–0.90 are typical for well-performing fraud classifiers on insurance data — this project's RF AUC of 0.84 falls squarely within this benchmark range.

Sahin and Duman (2011) apply decision trees and SVMs to credit card fraud, achieving fraud F1 scores of 0.43–0.56 on heavily imbalanced datasets — directly comparable to this project's fraud F1 range of 0.43–0.51. Sundarkumar and Ravi (2015) demonstrate that SMOTE-enhanced Random Forest achieves improved recall on insurance claim data, consistent with this project's application of SMOTE to balance the training set before RF training.

In the NLP and Transformer domain, Sanh et al. (2019) establish DistilBERT as an efficient and highly capable distillation of BERT for sequence classification tasks. While direct comparison with prior NLP-based insurance fraud work is limited by the novelty of the narrative-generation approach, the broader financial NLP literature (e.g., applications of BERT to earnings call fraud detection, loan application text analysis) supports the hypothesis that Transformer models can extract meaningful fraud signals from short structured texts. This project extends this direction by demonstrating the feasibility of a fully automated narrative-to-fraud-score pipeline applied to structured insurance claim tabular data.

4.6 Application Demonstration Analysis

The Streamlit application successfully operationalises all three experimental phases within a unified interface. Key technical features and their operational implications are:

- **Dynamic dataset loading:** Both local file and upload options provide flexibility for demonstration. @st.cache_resource prevents redundant model loading across interactions.
- **Bulletproof feature alignment:** Feature misalignment between training and inference is handled via feature_names_in_ alignment for RF models and pd.get_dummies() consistency for the hybrid model — critical for production robustness.
- **Real-time meta-learner training:** Experiment 3 trains the XGBoost meta-learner within the application using @st.cache_data, enabling reproducible live demonstration without requiring a pre-saved Experiment 3 model file.
- **Risk triage interactivity:** The tier filter (All/High/Medium/Low) simulates a genuine fraud operations workflow, enabling demonstration of probability-based prioritisation to non-technical stakeholders.
- **Graceful error handling:** Missing .pkl files produce informative error messages; unknown categorical values during inference are mapped to the most frequent known class, preventing runtime failures.

4.7 Discussion of Suitable Use Cases

The results and architecture developed in this project support four high-value real-world applications:

- **Claims Triage Automation:** The probability-tiered output directly supports fraud investigation teams in prioritising caseloads — High Risk claims (≥75% probability) are routed to specialist investigators while Low Risk claims are fast-tracked for payment, improving both investigative efficiency and customer experience for the majority of honest claimants.
- **Underwriting Risk Scoring:** The fraud probability score could augment underwriting models at policy inception, enabling proactive premium adjustments, enhanced verification requirements, or telematics mandates for high-risk profiles — shifting fraud detection upstream from the claims stage.
- **Regulatory Compliance Reporting:** The XAI feature importance outputs provide interpretable evidence of model decision factors, supporting compliance with the UK FCA's Consumer Duty obligations and GDPR Article 22 requirements for explainable automated decisions affecting customers.

- **Real-Time Claims Management Integration:** The DistilBERT + XGBoost pipeline can be containerised (Docker) and wrapped as a REST API endpoint, enabling integration with existing Claims Management Systems (CMS) to provide real-time fraud probability scores at the point of claim submission — without requiring unstructured text input from claimants.

5. Conclusions

5.1 Summary of Findings

This project has successfully developed, evaluated, and compared three experimental pipelines for insurance claim fraud detection across two real-world datasets. The primary findings are summarised as follows:

- **Experiment 1 (Fraud Oracle):** Random Forest and XGBoost establish tabular baselines on the large, severely imbalanced Fraud Oracle dataset (15,420 records, 6% fraud). Without SMOTE correction, both models exhibit high accuracy driven by majority class dominance. The experiment establishes the importance of appropriate evaluation metrics (F1, AUC) over accuracy for imbalanced fraud datasets.
- **Experiment 2 (Insurance Claims):** The Random Forest baseline achieves the strongest individual model performance (F1: 0.51, AUC: 0.84), outperforming the BiLSTM (F1: 0.44, AUC: 0.82) and the Hybrid RF+BiLSTM ensemble (F1: 0.46). The BiLSTM's underperformance is attributed to the small dataset size (1,000 records) and the absence of genuine sequential structure in the tabular feature representation. GridSearchCV optimisation of RF (n_estimators=100, max_depth=10, class_weight='balanced') produced an F1 of 0.43 — below the default — demonstrating that hyperparameter tuning can shift the precision-recall balance in non-obvious ways.
- **Experiment 3 (Hybrid NLP):** The DistilBERT + XGBoost hybrid architecture successfully demonstrates the feasibility of integrating NLP-derived semantic features with structured tabular data for fraud detection. Fine-tuning DistilBERT on structured claim narratives and using the resulting fraud probability as an engineered feature in XGBoost provides a novel and operationally deployable approach. The XAI feature importance analysis confirms that distilbert_fraud_score contributes meaningful predictive signal beyond the tabular features alone.
- **Application:** The Streamlit dashboard successfully operationalises all three experiments in a production-representative interactive interface, demonstrating real-time claim triage with probability-based risk tiering.

5.2 Answering the Research Question

"Can a hybrid architecture combining DistilBERT-derived semantic embeddings with XGBoost gradient boosting improve insurance claim fraud detection over tabular-only and deep learning-only baseline models?"

The evidence from this project provides a conditionally affirmative answer. The DistilBERT + XGBoost hybrid architecture is technically sound, operationally deployable, and demonstrably incorporates semantic NLP information (as validated by feature importance analysis) that is unavailable to purely tabular models. Definitive performance superiority over the Random Forest baseline in F1 and AUC metrics would require validation on substantially larger datasets (>10,000 records) where DistilBERT's semantic understanding can be more fully leveraged, and where the marginal contribution of the NLP score relative to richer tabular features can be more precisely isolated through ablation testing. On the 1,000-record Insurance Claims dataset, the hybrid approach demonstrates feasibility and motivates further investigation at scale.

5.3 Future Work

- **Larger-scale validation:** Deployment on datasets with 50,000+ records would provide statistically robust comparisons and allow deep learning and Transformer models to realise their full representational potential relative to gradient boosting.
- **Richer narrative templates:** Incorporating additional features (witness statements, adjuster notes, claim history, prior fraud flags) into the narrative template would generate richer textual representations and likely improve DistilBERT's fraud signal extraction.
- **Ablation study:** Systematically removing distilbert_fraud_score from the enriched feature set and measuring the resulting performance degradation would provide direct, quantified evidence of the NLP component's marginal contribution — strengthening the hybrid hypothesis beyond feature importance heuristics.
- **Domain-specific language models:** Fine-tuning FinBERT (Araci, 2019) or a domain-adapted insurance language model — rather than general-purpose DistilBERT — may extract stronger fraud signals from claim narratives.
- **SHAP explainability:** Integration of SHAP (SHapley Additive exPlanations; Lundberg and Lee, 2017) values would provide instance-level explanations for individual fraud decisions, directly supporting GDPR Article 22 compliance requirements for explainable automated decisions.
- **Federated learning:** Given GDPR constraints and the commercially sensitive nature of insurance claim data, federated learning — training models across distributed data sources without centralising records — represents a promising direction for privacy-preserving multi-insurer fraud detection collaboration.
- **Real-time API deployment:** Containerising the hybrid pipeline (Docker) and deploying as a cloud API (AWS SageMaker, Azure ML) would enable integration with production Claims Management Systems, providing real-time fraud probability scores at the point of claim submission.

References

Abdallah, A., Maarof, M.A. and Zainal, A. (2016) 'Fraud detection system: A survey', Journal of Network and Computer Applications, 68, pp. 90–113. doi:10.1016/j.jnca.2016.04.007.

ABI (2023) Insurance fraud statistics. Association of British Insurers. Available at: https://www.abi.org.uk/data-and-resources/tools-and-resources/claims-fraud/insurance-fraud-statistics (Accessed: [enter date]).

Abadi, M. et al. (2015) TensorFlow: Large-scale machine learning on heterogeneous systems [software]. Available at: https://www.tensorflow.org (Accessed: [enter date]).

Araci, D. (2019) 'FinBERT: Financial sentiment analysis with pre-trained language models', arXiv preprint arXiv:1908.10063.

Breiman, L. (2001) 'Random forests', Machine Learning, 45(1), pp. 5–32. doi:10.1023/A:1010933404324.

Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002) 'SMOTE: Synthetic minority over-sampling technique', Journal of Artificial Intelligence Research, 16, pp. 321–357.

Chen, T. and Guestrin, C. (2016) 'XGBoost: A scalable tree boosting system', Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, pp. 785–794.

Devlin, J., Chang, M.W., Lee, K. and Toutanova, K. (2018) 'BERT: Pre-training of deep bidirectional transformers for language understanding', arXiv preprint arXiv:1810.04805.

Goodfellow, I., Bengio, Y. and Courville, A. (2016) Deep learning. Cambridge, MA: MIT Press.

Hochreiter, S. and Schmidhuber, J. (1997) 'Long short-term memory', Neural Computation, 9(8), pp. 1735–1780.

Hunter, J.D. (2007) 'Matplotlib: A 2D graphics environment', Computing in Science & Engineering, 9(3), pp. 90–95.

Kaggle (2021a) Vehicle claim fraud detection [dataset]. Available at: https://www.kaggle.com/datasets/shivamb/vehicle-claim-fraud-detection (Accessed: [enter date]).

Kaggle (2021b) Auto insurance claims data [dataset]. Available at: https://www.kaggle.com/datasets/buntyshah/auto-insurance-claims-data (Accessed: [enter date]).

LeCun, Y., Bengio, Y. and Hinton, G. (2015) 'Deep learning', Nature, 521(7553), pp. 436–444.

Lemaître, G., Nogueira, F. and Aridas, C.K. (2017) 'Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning', Journal of Machine Learning Research, 18(17), pp. 1–5.

Lundberg, S.M. and Lee, S.I. (2017) 'A unified approach to interpreting model predictions', Advances in Neural Information Processing Systems, 30, pp. 4765–4774.

Pedregosa, F. et al. (2011) 'Scikit-learn: Machine learning in Python', Journal of Machine Learning Research, 12, pp. 2825–2830.

Phua, C., Lee, V., Smith, K. and Gayler, R. (2010) 'A comprehensive survey of data mining-based fraud detection research', arXiv preprint arXiv:1009.6119.

Sahin, Y. and Duman, E. (2011) 'Detecting credit card fraud by ANN and logistic regression', International Symposium on Innovations in Intelligent Systems and Applications, Istanbul: IEEE, pp. 315–319.

Sanh, V., Debut, L., Chaumond, J. and Wolf, T. (2019) 'DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter', arXiv preprint arXiv:1910.01108.

Schuster, M. and Paliwal, K.K. (1997) 'Bidirectional recurrent neural networks', IEEE Transactions on Signal Processing, 45(11), pp. 2673–2681.

Streamlit Inc. (2023) Streamlit: The fastest way to build data apps [software]. Available at: https://streamlit.io (Accessed: [enter date]).

Sundarkumar, G.G. and Ravi, V. (2015) 'A novel hybrid undersampling method for mining unbalanced datasets in banking and insurance', Engineering Applications of Artificial Intelligence, 37, pp. 368–377.

Vaswani, A. et al. (2017) 'Attention is all you need', Advances in Neural Information Processing Systems, 30, pp. 5998–6008.

Waskom, M.L. (2021) 'Seaborn: Statistical data visualization', Journal of Open Source Software, 6(60), p. 3021.

Wolf, T. et al. (2020) 'Transformers: State-of-the-art natural language processing', Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 38–45.

Appendices

Appendix A: Training Code – Tabular Models (1_train_tabular.py)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
import xgboost as xgb
import joblib
import os

# Set working directory to ensure files save in the right place
os.chdir(r"D:\python setup")

print("Loading data...")
df = pd.read_csv('insurance_claims.csv')

# Drop non-informative columns
cols_to_drop = ['_c39', 'policy_number', 'incident_date', 'policy_bind_date']
df = df.drop(columns=[c for c in cols_to_drop if c in df.columns])

# Separate features and target
target_col = 'fraud_reported'
X = df.drop(columns=[target_col])
y = df[target_col].apply(lambda x: 1 if x == 'Y' else 0)

# Encode categorical variables and save encoders
print("Encoding features...")
label_encoders = {}
for col in X.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    X[col] = X[col].astype(str)
    X[col] = le.fit_transform(X[col])
    label_encoders[col] = le

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Train Random Forest
print("Training Random Forest...")
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Train XGBoost
print("Training XGBoost...")
xgb_model = xgb.XGBClassifier(n_estimators=100, max_depth=4, random_state=42,
eval_metric='logloss')
xgb_model.fit(X_train, y_train)

# Save everything for the Streamlit dashboard
print("Saving tabular models...")
joblib.dump(rf_model, 'rf_model_2.pkl')
joblib.dump(xgb_model, 'xgb_model_2.pkl')
joblib.dump(label_encoders, 'encoders_2.pkl')

print("✅ Tabular setup complete! Proceed to step 2.")
```

Appendix B: Training Code – DistilBERT Fine-Tuning (2_train_distilbert.py)

```
import pandas as pd
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer,
TrainingArguments
from datasets import Dataset
import os
```

```
os.chdir(r"D:\python setup")
MODEL_DIR = r"D:\python setup\distilbert_claims_model"

def build_claim_narrative(row):
    return f"A {row['age']}-year-old customer from {row['policy_state']} reported a
{row['incident_severity']} {row['incident_type']} incident. The authorities were
{row['police_report_available']} to the scene. The total requested claim amount is
${row['total_claim_amount']}, which includes a vehicle damage claim of
${row['vehicle_claim']}."

print("Preparing text data...")
df = pd.read_csv('insurance_claims.csv')
df['text'] = df.apply(build_claim_narrative, axis=1)
df['label'] = df['fraud_reported'].apply(lambda x: 1 if x == 'Y' else 0)

# Convert to Hugging Face Dataset
dataset = Dataset.from_pandas(df[['text', 'label']])
dataset = dataset.train_test_split(test_size=0.2, seed=42)

print("Loading tokenizer and base model...")
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
model = AutoModelForSequenceClassification.from_pretrained("distilbert-base-uncased",
num_labels=2)

def tokenize_function(examples):
    return tokenizer(examples["text"], padding="max_length", truncation=True,
max_length=128)

tokenized_datasets = dataset.map(tokenize_function, batched=True)

training_args = TrainingArguments(
    output_dir="./hf_results",
    num_train_epochs=3,
    per_device_train_batch_size=16,
    save_strategy="no", # We handle saving manually at the end
    logging_steps=10
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["test"],
)

print("Training DistilBERT (This may take a few minutes)...")
trainer.train()

print(f"Saving model directly to {MODEL_DIR}...")
model.save_pretrained(MODEL_DIR)
tokenizer.save_pretrained(MODEL_DIR)

print("✅ DistilBERT trained and saved successfully! Proceed to step 3.")
```

Appendix C: Hybrid Score Generation (3_generate_hybrid.py)

```
import pandas as pd
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch.nn.functional as F
import os

os.chdir(r"D:\python setup")
MODEL_DIR = r"D:\python setup\distilbert_claims_model"
```

```
def build_claim_narrative(row):
    return f"A {row['age']}-year-old customer from {row['policy_state']} reported a
{row['incident_severity']} {row['incident_type']} incident. The authorities were
{row['police_report_available']} to the scene. The total requested claim amount is
${row['total_claim_amount']}, which includes a vehicle damage claim of
${row['vehicle_claim']}."

print("Loading original dataset...")
df = pd.read_csv('insurance_claims.csv')
df['text'] = df.apply(build_claim_narrative, axis=1)

print(f"Loading local DistilBERT model from {MODEL_DIR}...")
# The 'r' before the string ensures Windows paths are read correctly
tokenizer = AutoTokenizer.from_pretrained(MODEL_DIR)
model = AutoModelForSequenceClassification.from_pretrained(MODEL_DIR)
model.eval()

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

print("Scoring claims (Generating deep learning probabilities)...")
dl_scores = []

# Process in small batches so we don't run out of memory
for text in df['text'].tolist():
    inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True,
max_length=128).to(device)
    with torch.no_grad():
        outputs = model(**inputs)
        probs = F.softmax(outputs.logits, dim=-1)
        # Get the probability of class 1 (Fraud)
        fraud_prob = probs[0][1].item()
        dl_scores.append(fraud_prob)

# Attach scores and clean up text column
df['distilbert_fraud_score'] = dl_scores
df = df.drop(columns=['text'])

# Convert target back to 1s and 0s for XGBoost compatibility
df['fraud_reported'] = df['fraud_reported'].apply(lambda x: 1 if x == 'Y' else 0)

print("Saving enriched dataset...")
df.to_csv('insurance_claims_enriched.csv', index=False)

print("✅ Pipeline complete! Your Streamlit app is ready to run.")
```

Appendix D: Deep Learning Score Generation (generate_dl_scores.py)

```
import pandas as pd

# 1. Load the preprocessed 1,000-row dataset
df = pd.read_csv('insurance_claims.csv')

# 2. Define the narrative template
def build_claim_narrative(row):
    """
    Translates a single row of tabular claims data into a natural language string.
    Note: Adjust the column names below to match your exact dataframe structure.
    """
    narrative = (
        f"A {row['age']}-year-old customer from {row['policy_state']} reported a "
        f"{row['incident_severity']} {row['incident_type']} incident. "
        f"The authorities were {row['police_report_available']} to the scene. "
        f"The total requested claim amount is ${row['total_claim_amount']}, "
        f"which includes a vehicle damage claim of ${row['vehicle_claim']}."
```

```
    )
    return narrative

# 3. Apply the function to create a new text feature
df['claim_narrative'] = df.apply(build_claim_narrative, axis=1)

# Preview the first narrative to ensure the conversion worked
print(df['claim_narrative'].iloc[0])

import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from scipy.special import softmax

# 1. Load your fine-tuned Hugging Face model and tokenizer
# Point this path to wherever your Trainer class saved the final weights
# Use an absolute path to be completely safe
model_path = "D:/python setup/distilbert_claims_model"

tokenizer = AutoTokenizer.from_pretrained(model_path)
model = AutoModelForSequenceClassification.from_pretrained(model_path)

# Set the model to evaluation mode (disables dropout layers)
model.eval()

# 2. Define the scoring function
def extract_fraud_probability(text):
    # Tokenize the narrative text
    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        padding=True,
        max_length=512
    )

    # Run the forward pass without tracking gradients (saves memory)
    with torch.no_grad():
        outputs = model(**inputs)
        # Extract the raw logits for the classes
        logits = outputs.logits.numpy()[0]

    # Apply softmax to convert logits to probability percentages
    probabilities = softmax(logits)

    # Assuming Class 0 is 'Genuine' and Class 1 is 'Fraud'
    fraud_prob = probabilities[1]
    return fraud_prob

# 3. Generate the deep learning scores
# For 1,000 rows, a standard pandas apply is efficient enough.
df['distilbert_fraud_score'] = df['claim_narrative'].apply(extract_fraud_probability)

# 4. Save the enriched dataset for the XGBoost phase
df.drop(columns=['claim_narrative'], inplace=True) # Drop text to keep it purely
tabular
df.to_csv('insurance_claims_enriched.csv', index=False)

print("DistilBERT scoring complete. Dataset ready for XGBoost.")

# Save the fine-tuned model and tokenizer to a specific folder
trainer.save_model("D:/python setup/distilbert_claims_model")
tokenizer.save_pretrained("D:/python setup/distilbert_claims_model")
```

Appendix E: Streamlit Application Code (app.py)

```
import streamlit as st
import pandas as pd
```

```
import joblib
import xgboost as xgb
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score, precision_score,
recall_score

# 1. Page Configuration
st.set_page_config(page_title="Fraud Detection Pipeline", layout="wide")

# 2. Sidebar Navigation
st.sidebar.title("Dissertation Experiments")
experiment = st.sidebar.radio(
    "Select Phase to Analyze:",
    (
        "Experiment 1: Fraud Oracle",
        "Experiment 2: Insurance Claims",
        "Experiment 3: Hybrid Model (DistilBERT + XGBoost)"
    )
)

# ========================================
# EXPERIMENTS 1 & 2: TABULAR COMPARISONS
# ========================================
if experiment in ["Experiment 1: Fraud Oracle", "Experiment 2: Insurance Claims"]:

    st.title(f"Tabular Baselines: {experiment.split(':')[1]}")
    st.write("Compare standard machine learning architectures without NLP text
enrichment.")

    # 3. Dynamic Model Loading
    @st.cache_resource
    def load_resources(exp_choice):
        if exp_choice == "Experiment 1: Fraud Oracle":
            rf = joblib.load('random_forest.pkl')
            xgb_model = joblib.load('xgboost_model.pkl')
            le_dict = joblib.load('label_encoders.pkl')
            target_col = 'FraudFound_P'
        else:
            rf = joblib.load('rf_model_2.pkl')
            xgb_model = joblib.load('xgb_model_2.pkl')
            le_dict = joblib.load('encoders_2.pkl')
            target_col = 'fraud_reported'

        return rf, xgb_model, le_dict, target_col

    try:
        rf_model, xgb_model, label_encoders, target_col = load_resources(experiment)
    except FileNotFoundError:
        st.error(f"Error: The .pkl files for {experiment} were not found in your
directory.")
        st.stop()

    # 4. Data Loading Options (Bug Fixed with unique keys)
    st.markdown("---")

    use_local = st.checkbox(
        f"Use local {experiment.split(':')[1]} dataset automatically",
        key=f"check_{experiment}" # Unique key prevents state bleed
    )

    uploaded_file = st.file_uploader(
        f"Or upload a custom CSV file",
        type=['csv'],
        key=f"upload_{experiment}" # Unique key prevents state bleed
    )

    df = None
```

```
    if use_local:
        try:
            file_path = 'insurance_claims.csv' if "Insurance Claims" in experiment else
'fraud_oracle.csv'
            df = pd.read_csv(file_path)
        except FileNotFoundError:
            st.error(f"⚠️ Could not find '{file_path}'. Please check your folder.")
    elif uploaded_file is not None:
        df = pd.read_csv(uploaded_file)


    if df is not None:
        display_df = df.copy()

        # 1. Drop known identifiers and dates
        cols_to_drop = [target_col, '_c39', 'incident_date', 'policy_bind_date',
'policy_number']
        X = df.drop(columns=[c for c in cols_to_drop if c in df.columns],
errors='ignore')

        # 2. Preprocess Data
        for col in list(X.columns):
            if col in label_encoders:
                le = label_encoders[col]
                X[col] = X[col].fillna('Unknown').astype(str)
                known_classes = set(le.classes_)
                X[col] = X[col].apply(lambda x: x if x in known_classes else
le.classes_[0])
                X[col] = le.transform(X[col])
            elif X[col].dtype == 'object':
                X = X.drop(col, axis=1)

        X = X.fillna(0)

        # 3. Bulletproof Feature Alignment
        if hasattr(rf_model, 'feature_names_in_'):
            X = X[rf_model.feature_names_in_]

        # 4. Make Predictions
        try:
            rf_preds = rf_model.predict(X)
            xgb_preds = xgb_model.predict(X)

            results_df = display_df.copy()
            results_df['RF_Prediction'] = ["Fraud" if p == 1 else "Genuine" for p in
rf_preds]
            results_df['XGB_Prediction'] = ["Fraud" if p == 1 else "Genuine" for p in
xgb_preds]
            results_df['Models_Match'] = results_df['RF_Prediction'] ==
results_df['XGB_Prediction']

            st.subheader("Analysis Results")
            col1, col2, col3 = st.columns(3)
            col1.metric("Total Claims Analyzed", len(results_df))
            col2.metric("RF Fraud Detections", sum(rf_preds))
            col3.metric("XGB Fraud Detections", sum(xgb_preds))

            st.dataframe(results_df[['RF_Prediction', 'XGB_Prediction', 'Models_Match']
+ [c for c in display_df.columns]])

        except ValueError as e:
            st.error(f"Prediction Error: Feature mismatch. Details: {e}")


# ========================================
# EXPERIMENT 3: HYBRID ARCHITECTURE (MSc Level)
# ========================================
else:
    st.title("Experiment 3: Hybrid AI Architecture")
    st.write("This phase demonstrates the dissertation's core hypothesis: enriching
tabular gradient boosting models with Deep Learning (DistilBERT) textual embeddings
yields superior fraud detection.")
    st.markdown("---")
```

```
    @st.cache_data
    def load_hybrid_data():
        try:
            return pd.read_csv('insurance_claims_enriched.csv')
        except FileNotFoundError:
            return None

    df_hybrid = load_hybrid_data()

    if df_hybrid is None:
        st.info("ℹ️ The Hybrid Architecture requires the
'insurance_claims_enriched.csv' file generated by the DistilBERT pipeline (Script 2).
Please run that script first.")
    else:
        # Prepare Data
        target_col = 'fraud_reported'
        X = df_hybrid.drop(columns=[target_col, '_c39'], errors='ignore')
        y = df_hybrid[target_col]

        X = X.fillna(0)
        X = pd.get_dummies(X, drop_first=True)

        X_train, X_test, y_train, y_test, indices_train, indices_test =
train_test_split(
            X, y, df_hybrid.index, test_size=0.2, random_state=42
        )

        # Train Meta-Learner
        with st.spinner("Compiling Hybrid Meta-Learner..."):
            hybrid_model = xgb.XGBClassifier(n_estimators=100, max_depth=4,
random_state=42, eval_metric='logloss')
            hybrid_model.fit(X_train, y_train)

        # Get Predictions AND Probabilities
        y_pred = hybrid_model.predict(X_test)
        y_prob = hybrid_model.predict_proba(X_test)[:, 1] # Probability of Fraud

        # Calculate Metrics
        acc = accuracy_score(y_test, y_pred)
        prec = precision_score(y_test, y_pred, zero_division=0)
        rec = recall_score(y_test, y_pred, zero_division=0)

        # Build Interactive UI
        tab1, tab2, tab3 = st.tabs(["📊 Performance Metrics", "🧠 Explainable AI
(XAI)", "🚨 Actionable Insights"])

        with tab1:
            st.subheader("Global Model Performance")
            col1, col2, col3 = st.columns(3)
            col1.metric("Overall Accuracy", f"{acc * 100:.1f}%", "Hybrid Edge")
            col2.metric("Precision (Quality)", f"{prec * 100:.1f}%")
            col3.metric("Recall (Capture Rate)", f"{rec * 100:.1f}%")

            st.text("Detailed Classification Report:")
            st.code(classification_report(y_test, y_pred))


        with tab2:
            st.subheader("Feature Importance & NLP Value")
            st.write("This chart isolates the features that drive the Hybrid Model's
decision-making. Look for the DistilBERT probability/score feature to prove the value
of textual analysis.")

            importance = hybrid_model.feature_importances_
            feat_imp_df = pd.DataFrame({'Feature': X.columns, 'Importance':
importance})
            feat_imp_df = feat_imp_df.sort_values(by='Importance',
ascending=False).head(12)

            # Display sorted bar chart
            st.bar_chart(feat_imp_df.set_index('Feature'))
```

```
        with tab3:
            st.subheader("Claims Triage & Risk Queue")
            st.write("By extracting the **probability scores** from the XGBoost
meta-learner, we can triage claims from lowest to highest risk, providing actionable
intelligence rather than just binary labels.")

            # Create a nice output dataframe
            results_df = df_hybrid.loc[indices_test].copy()
            results_df['Fraud_Probability'] = np.round(y_prob * 100, 1)

            # Categorize Risk
            conditions = [
                (results_df['Fraud_Probability'] >= 75),
                (results_df['Fraud_Probability'] >= 40) &
(results_df['Fraud_Probability'] < 75),
                (results_df['Fraud_Probability'] < 40)
            ]
            choices = ['🔴 High Risk', '🟡 Medium Risk', '🟢 Low Risk']
            results_df['Risk_Tier'] = np.select(conditions, choices, default='Unknown')

            # Interactive Filter
            selected_tier = st.selectbox(
                "Filter by Risk Tier:",
                ["View All", "🔴 High Risk", "🟡 Medium Risk", "🟢 Low Risk"]
            )

            # Apply the filter
            if selected_tier == "View All":
                display_df = results_df.sort_values(by='Fraud_Probability',
ascending=False)
            else:
                display_df = results_df[results_df['Risk_Tier'] ==
selected_tier].sort_values(by='Fraud_Probability', ascending=False)

            # Show how many claims match the filter
            st.metric(f"Total Claims ({selected_tier})", len(display_df))

            # Reorder columns to show the cool stuff first
            cols_to_show = ['Risk_Tier', 'Fraud_Probability', target_col] + [c for c in
display_df.columns if c not in ['Risk_Tier', 'Fraud_Probability', target_col, '_c39']]
            st.dataframe(display_df[cols_to_show])
```